Q

# MealQ

## เว็บแอปสั่งอาหาร และจัดการคิวรับอาหาร

Node.js, Express และ MongoDB

การประยุกต์บทเรียน Web Application



# ภาพรวมระบบและบทบาทผู้ใช้

ลูกค้าสั่งอาหาร ร้านค้าจัดคิว และแอดมินดูแลร้านกับผู้ใช้

| ส่วนของระบบ | เทคโนโลยี | หน้าที่ |
| --- | --- | --- |
| Browser | React, HTML/CSS, JavaScript | แสดงร้าน เมนู ตะกร้า และสถานะ |
| API Server | Node.js, Express, Mongoose | รับคำขอ ตรวจสิทธิ์ และกฎสั่งซื้อ |
| Database | MongoDB | เก็บผู้ใช้ ร้าน เมนู ออเดอร์ และสต็อก |

**HTTP/JSON เชื่อม Browser กับ API ส่วน Mongoose เชื่อม API กับ MongoDB**

# HTML และ CSS ในหน้าจอ MealQ

พื้นฐานโครงสร้างและรูปแบบของหน้าเว็บ

## HTML

ใช้ฟอร์ม ช่องกรอก รูปภาพ และรายการ
ในหน้าเมนู ตะกร้า และชำระเงิน

## CSS และ Tailwind

กำหนดสี ตัวอักษร และระยะห่าง
ให้ชื่ออาหาร ราคา และปุ่มอ่านง่าย

## Responsive layout

ใช้ Grid/Flex และ breakpoint
จัดหน้ามือถือกับคอมพิวเตอร์



ภาพเมนูที่ใช้ในแอป Demo

# JavaScript กับข้อมูลในระบบ

นำ Array, Object และ Function มาใช้กับรายการอาหาร

| สิ่งที่เรียน | ใช้ใน MealQ |
| --- | --- |
| map() | แสดงรายการอาหารและจัดรูปแบบข้อมูลออเดอร์ |
| filter() | กรองประเภทอาหาร งบประมาณ และรายการที่ขายได้ |
| reduce() | รวมยอดอาหารในตะกร้า |
| Math.random() / Math.floor() | สุ่มเมนูจากรายการที่ผ่านตัวกรอง |

ตัวอย่างคำนวณยอดอาหารจากโค้ดจริง

```
lines.reduce((sum, l) => sum + round(l.lineTotal), 0)
```

# Event และ DOM ในการโต้ตอบกับผู้ใช้

รับเหตุการณ์ ตรวจข้อมูล และแสดงผลตามสถานะ

## onChange

เปลี่ยนวิธีชำระเงินและตัวเลือกอาหาร

## onSubmit + preventDefault()

ตรวจฟอร์มก่อนส่งออเดอร์
แล้วเปลี่ยนหน้าเมื่อข้อมูลพร้อม

createRoot(document.getElementById('root'))

จุดเชื่อม HTML กับการแสดงผลของ React ใน main.jsx

## React state

แสดงสถานะกำลังส่ง
สำเร็จ หรือเกิดข้อผิดพลาด
ตามผลที่ได้รับจาก API

# HTTP และ Async เชื่อมหน้าเว็บกับ API

Request–Response, JSON และการจัดการข้อผิดพลาด

## ฝั่ง Browser

fetch + async/await
ส่งและรอข้อมูลโดยไม่หยุดทั้งหน้า

JSON.stringify() ส่งข้อมูล
response.json() อ่านผลตอบกลับ

## ฝั่ง Express API

**GET /api/v1/restaurants**

อ่านข้อมูลร้านอาหาร

**POST /api/v1/orders**

ตรวจรายการและสร้างออเดอร์

**201 สร้างสำเร็จ** **401 ยังไม่เข้าสู่ระบบ** **409 วัตถุดิบไม่พอ**

# Node.js Modules และ Express

แบ่งโค้ดฝั่งเซิร์ฟเวอร์และตรวจสิทธิ์ก่อนทำงาน

## CommonJS Modules

```
const express = require("express");
module.exports = router;
```

แยก Routes, Controllers,
Services และ Models

## Express Middleware

```
requireAuth
requireRole("customer")
```

ตรวจ session และสิทธิ์
ก่อนเรียก Controller

## POST /api/v1/orders

```
router.post("/", asyncHandler(controller.createOrder));
```

# MongoDB และ Mongoose

ต่อยอดจาก Object/JSON สู่ข้อมูลที่บันทึกและค้นคืนได้

| ข้อมูล | บันทึกอะไร | ใช้ในส่วนไหน |
| --- | --- | --- |
| User / Store | บัญชี บทบาท และข้อมูลร้าน | เข้าสู่ระบบและแสดงร้าน |
| Menu / Ingredient | ราคา สูตร และจำนวนวัตถุดิบ | จัดการเมนูและตรวจสต็อก |
| Order | รายการ ยอดเงิน และสถานะ | ติดตามและค้นประวัติ |

**Mongoose กำหนด Schema, validation และความสัมพันธ์ด้วย ObjectId**

CRUD เมนู: เพิ่ม (POST), อ่าน (GET), แก้ไข (PATCH), ลบ (DELETE)

# การออกแบบความสัมพันธ์ของข้อมูล

ObjectId เชื่อมเอกสาร ส่วนสูตรและรายการสั่งซื้อเก็บเป็นข้อมูลย่อย

| ความสัมพันธ์ | จำนวน | ฟิลด์อ้างอิง |
|---|---|---|
| Store กับ Menu | 1 : หลาย | Menu.store_id |
| Store กับ Ingredient | 1 : หลาย | Ingredient.store_id |
| User (ลูกค้า) กับ Order | 1 : หลาย | Order.customer_id |
| Store กับ Order | 1 : หลาย | Order.store_id |
| Menu กับ Ingredient | หลาย : หลาย | Menu.ingredients[].ingredient_id |

**สูตรใน Menu.ingredients[] เก็บ ingredient_id, quantity และ unit**

Order.items[] เก็บชื่อ ราคา และจำนวน ณ เวลาสั่ง เพื่อคงประวัติเดิม

# ตัวอย่างการสั่งซื้อจนรับอาหาร

**01** **ลูกค้าเลือกอาหารและยืนยัน**

React และ JavaScript จัดรายการ แล้วส่งผ่าน Fetch API

**02** **ตรวจสูตรและวัตถุดิบก่อนหักเงิน**

Express ตรวจจำนวนและรวมวัตถุดิบของทั้งตะกร้า

**03** **บันทึกออเดอร์และตัดสต็อก**

MongoDB เก็บรายการและปรับจำนวนวัตถุดิบตามสูตร

**04** **ร้านทำอาหารและตรวจรหัสรับ**

เปลี่ยนสถานะจนพร้อมรับ ตรวจรหัส 4 หลักหรือ QR แล้วจบออเดอร์

**mealq.site/demo**

Wallet / PromptPay เป็นโหมด Demo